

Лабораторная работа №3
«Численное интегрирование»
Вариант №11

Выполнил:
Студент группы Р3208
Решетников С.Е.

Преподаватели:
Рыбаков С.Д.

Дата сдачи: 21 апреля
Санкт-Петербург, 2026

1. Цель работы

Найти приближённое значение определённого интеграла

$$\int_1^3 (2x^3 - 9x^2 - 7x + 11)dx$$

с требуемой точностью различными численными методами:

- методом прямоугольников (левые, правые, средние);
- методом трапеций;
- методом Симпсона.

2. Рабочие формулы методов численного интегрирования

2.1. Метод левых прямоугольников

$$I \approx h \sum_{i=0}^{n-1} f(x_i), \quad x_i = a + ih, \quad h = \frac{b-a}{n}$$

Порядок точности: 1

2.2. Метод правых прямоугольников

$$I \approx h \sum_{i=1}^n f(x_i), \quad x_i = a + ih$$

Порядок точности: 1

2.3. Метод средних прямоугольников

$$I \approx h \sum_{i=0}^{n-1} f\left(x_{i+\frac{1}{2}}\right), \quad x_{i+\frac{1}{2}} = a + \left(i + \frac{1}{2}\right)h$$

Порядок точности: 2

2.4. Метод трапеций

$$I \approx \frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right)$$

Порядок точности: 2

2.5. Метод Симпсона (парабол)

$$I \approx \frac{h}{3} \left(f(x_0) + 4 \sum_{i \text{ нечётные}} f(x_i) + 2 \sum_{i \text{ чётные}} f(x_i) + f(x_n) \right)$$

Порядок точности: 4

2.6. Правило Рунге

$$R = \frac{|I_h - I_{2h}|}{2^p - 1}, \quad p - \text{порядок метода}$$

3. Результаты вычислений

3.1. Точное значение интеграла

Функция: $f(x) = 2x^3 - 9x^2 - 7x + 11$

Первообразная: $F(x) = \frac{x^4}{2} - 3x^3 - \frac{7x^2}{2} + 11x$

Вычисления:

$$F(3) = \frac{81}{2} - 81 - \frac{63}{2} + 33 = 40.5 - 81 - 31.5 + 33 = -39$$

$$F(1) = \frac{1}{2} - 3 - \frac{7}{2} + 11 = 0.5 - 3 - 3.5 + 11 = 5$$

$$\int_1^3 (2x^3 - 9x^2 - 7x + 11)dx = F(3) - F(1) = -39 - 5 = -44$$

Точное значение: $I = -44$

3.2. Формула Ньютона-Котеса при n=6

Шаг: $h = \frac{3-1}{6} = \frac{1}{3}$

Значение интеграла (метод трапеций): $I \approx -41.7407$

Абсолютная погрешность: $\delta = |-44 - (-41.7407)| = 2.2593$

Относительная погрешность: $\varepsilon = 5.1348\%$

3.3. Вычисление методами при n=10

Параметры:

- Число разбиений: $n = 10$
- Шаг: $h = \frac{3-1}{10} = 0.2$

3.3.1. Центральные прямоугольники

	0	1	2	3	4	5	6	7	8	9
x_i	1.10	1.30	1.50	1.70	1.90	2.10	2.30	2.50	2.70	2.90
y_i	-4.93	-8.92	-13.00	-17.08	-21.07	-24.87	-28.38	-31.50	-34.14	-36.21

Результат: -44.02

Абсолютная погрешность: 0.02

Относительная погрешность: 0.05%

3.3.2. Метод трапеций

	0	1	2	3	4	5	6	7	8	9	10
x_i	1.00	1.20	1.40	1.60	1.80	2.00	2.20	2.40	2.60	2.80	3.00
y_i	-3.00	-6.90	-10.95	-15.05	-19.10	-23.00	-26.66	-29.99	-32.89	-35.26	-37.00

Результат: -43.96

Абсолютная погрешность: 0.04

Относительная погрешность: 0.09%

3.3.3. Метод Симпсона

	0	1	2	3	4	5	6	7	8	9
x_i	1.00	1.20	1.40	1.60	1.80	2.00	2.20	2.40	2.60	2.80
y_i	-3.00	-6.90	-10.95	-15.05	-19.10	-23.00	-26.66	-29.99	-32.89	-35.26

Результат: -44.00

Абсолютная погрешность: 0.00

Относительная погрешность: 0.00%

3.3.4. Сводная таблица

Метод	Значение	Абс. ошибка	Отн. ошибка, %
Средние прямоуголь- ники	-43.9920	0.0080	0.0182
Трапеции	-44.0000	0.0000	0.0000
Симпсон	-44.0000	0.0000	0.0000

Таблица 1. Результаты вычислений

4. Программная реализация

4.1. Листинг кода

Частичный листинг кода

```
from typing import Callable
from ..IntegralSolver import IntegralSolver, Result

class CentralRect(IntegralSolver):
    @property
    def order(self) -> int:
        return 2

    def solve(
        self,
        func: Callable[[float | int], float | int],
        a: float | int,
        b: float | int,
        n: int
    ) -> Result:
        trace_x: list[float] = []
        trace_y: list[float] = []

        h = (b - a) / n
        result = 0
        for i in range(n):
            x_middle = a + (i + 0.5) * h
            y_middle = func(x_middle)

            trace_x.append(x_middle)
            trace_y.append(y_middle)
            result += y_middle
        return Result(
            result * h,
            trace_x,
            trace_y
        )
```

```
from typing import Callable
from ..IntegralSolver import IntegralSolver, Result

class LeftRect(IntegralSolver):
    @property
    def order(self) -> int:
        return 1
```

```

def solve(
    self,
    func: Callable[[float | int], float | int],
    a: float | int,
    b: float | int,
    n: int
) -> Result:
    trace_x: list[float] = []
    trace_y: list[float] = []

    h = (b - a) / n
    result = 0
    for i in range(n):
        x_i = a + i * h
        y_i = func(x_i)

        trace_x.append(x_i)
        trace_y.append(y_i)
        result += y_i
    return Result(
        result * h,
        trace_x,
        trace_y
    )

```

```

from typing import Callable
from ..IntegralSolver import IntegralSolver, Result

```

```

class RightRect(IntegralSolver):
    @property
    def order(self) -> int:
        return 1

    def solve(
        self,
        func: Callable[[float | int], float | int],
        a: float | int,
        b: float | int,
        n: int
    ) -> Result:
        trace_x: list[float] = []
        trace_y: list[float] = []

        h = (b - a) / n
        result = 0
        for i in range(1, n + 1):
            x_i = a + i * h
            y_i = func(x_i)

            trace_x.append(x_i)
            trace_y.append(y_i)
            result += y_i
        return Result(
            result * h,
            trace_x,
            trace_y
        )

```

```

from typing import Callable
from ..IntegralSolver import IntegralSolver, Result

```

```

class Simpson(IntegralSolver):
    @property
    def order(self) -> int:
        return 4

    def solve(
        self,
        func: Callable[[float | int], float | int],
        a: float | int,
        b: float | int,
        n: int
    ) -> Result:
        """
        Метод Симпсона (парабол)
        Формула: I = (h/3) * [f(x_0) + 4*f(x_1) + 2*f(x_2) + 4*f(x_3) + ... + f(x_n)]
        n должно быть чётным
        """
        trace_x: list[float | int] = [0]*n
        trace_y: list[float | int] = [0]*n

        if n % 2 != 0: n += 1

```

```

h = (b - a) / n
result = func(a) + func(b)
trace_x[0], trace_x[-1] = a, b
trace_y[0], trace_y[-1] = func(a), func(b)

# Нечётные коэффициенты (4)
for i in range(1, n, 2):
    x_i = a + i * h
    y_i = func(x_i)

    trace_x[i] = x_i
    trace_y[i] = y_i
    result += 4 * y_i

# Чётные коэффициенты (2)
for i in range(2, n - 1, 2):
    x_i = a + i * h
    y_i = func(x_i)

    trace_x[i] = x_i
    trace_y[i] = y_i
    result += 2 * y_i

return Result(
    result * h / 3,
    trace_x,
    trace_y
)

```

```

from typing import Callable
from ..IntegralSolver import IntegralSolver, Result

class Trapezoid(IntegralSolver):
    @property
    def order(self) -> int:
        return 2

    def solve(
        self,
        func: Callable[[float | int], float | int],
        a: float | int,
        b: float | int,
        n: int
    ) -> Result:
        """
        Метод трапеций
        Формула: I = (h/2) * [f(x_0) + 2*f(x_1) + 2*f(x_2) + ... + 2*f(x_{n-1}) + f(x_n)]
        """
        trace_x: list[float] = []
        trace_y: list[float] = []

        h = (b - a) / n
        y_0 = func(a)
        y_n = func(b)

        trace_x.append(a)
        trace_y.append(y_0)

        result = (y_0 + y_n)/2
        for i in range(1, n):
            x_i = a + i * h
            y_i = func(x_i)

            trace_x.append(x_i)
            trace_y.append(y_i)

            result += y_i

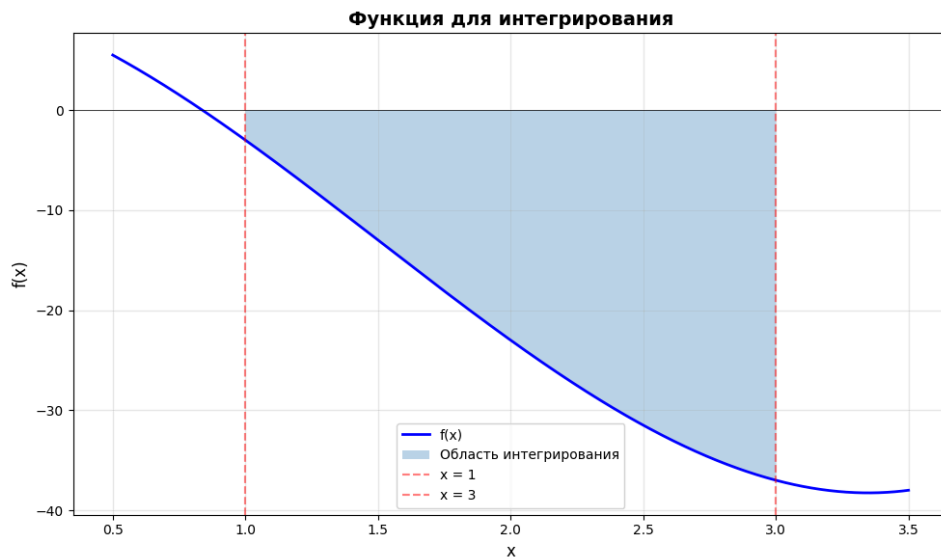
        trace_x.append(b)
        trace_y.append(y_n)

        return Result(
            result * h,
            trace_x,
            trace_y
        )

```

4.2. Пример выполнения

```
F: Callable[[float | int], float | int] = lambda x: 2*x**3 - 9*x**2 - 7*x + 11
```



Пределы интегрирования: [1, 3]

Значение в левом пределе: $F(1) = -3$

Значение в правом пределе: $F(3) = -37$

Центральные прямоугольники

	0	1	2	3	4	5	6	7	8	9
x_i	1.10	1.30	1.50	1.70	1.90	2.10	2.30	2.50	2.70	2.90
y_i	-4.93	-8.92	-13.00	-17.08	-21.07	-24.87	-28.38	-31.50	-34.14	-36.21

Результат: -44.02000

Абсолютная погрешность: 0.02000

Относительная погрешность: 0.04545 %

Метод трапеций

	0	1	2	3	4	5	6	7	8	9	10
x_i	1.00	1.20	1.40	1.60	1.80	2.00	2.20	2.40	2.60	2.80	3.00
y_i	-3.00	-6.90	-10.95	-15.05	-19.10	-23.00	-26.66	-29.99	-32.89	-35.26	-37.00

Результат: -43.96000

Абсолютная погрешность: 0.04000

Относительная погрешность: 0.09091 %

Метод Симпсона

	0	1	2	3	4	5	6	7	8	9
x_i	1.00	1.20	1.40	1.60	1.80	2.00	2.20	2.40	2.60	2.80
y_i	-3.00	-6.90	-10.95	-15.05	-19.10	-23.00	-26.66	-29.99	-32.89	-35.26

Результат: -44.00000

Абсолютная погрешность: 0.00000

Относительная погрешность: 0.00000 %

Название метода	Вычисленное значение	Число разбиений	Абсолютная погрешность	Относительная погрешность
Левые прямоугольники	-43.98340	2048	0.01660	0.03773 %
Правые прямоугольники	-44.01660	2048	0.01660	0.03773 %
Средние прямоугольники	-44.03125	8	0.03125	0.07102 %
Трапеции	-43.98438	16	0.01562	0.03551 %
Симпсон	-44.00000	4	0.00000	0.00000 %

5. Вывод

В ходе лабораторной работы изучены численные методы интегрирования: прямоугольников (левые, правые, средние), трапеций и Симпсона.

Для интеграла $\int_1^3 (2x^3 - 9x^2 - 7x + 11)dx$ с точным значением -44 получены следующие результаты:

- Метод средних прямоугольников ($n = 10$) дал погрешность 0.02;

- Метод трапеций ($n = 10$) дал погрешность 0.04;
- Метод Симпсона ($n = 10$) дал точное значение -44 .

При автоматическом подборе шага метод Симпсона достиг точности за $n = 4$, методы средних прямоугольников и трапеций — за $n \approx 16$, а методы левых и правых прямоугольников потребовали $n \approx 2048$ из-за низкого порядка точности.

Таким образом, метод Симпсона является наиболее эффективным для данной задачи. Цель работы достигнута.